TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Unidad 4
Librerías, plugins y permisos

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Prof. Titular: Lic. Franco Ruis Dias · JTP: Lic. Germán Cueva

Expo SDK 57 · React Native 0.86 · React 19.2

UNIDAD 4

Qué vamos a ver

Objetivos

- Entender cómo el código JavaScript llega al hardware del teléfono.
- Elegir e instalar una librería nativa sin romper el proyecto.
- Configurar el proyecto nativo sin abrir `android/` ni `ios/`.
- Declarar y pedir permisos como corresponde en cada plataforma.

Partes

- De JavaScript al hardware
- Generación nativa continua y config plugins
- Permisos

Las APIs concretas —cámara, ubicación, sensores, archivos— van en el deck siguiente.

UNIDAD 4 · PARTE 1

De JavaScript al hardware

Qué hay entre `import` y la cámara del teléfono.

Lo que cambia a partir de acá

UNIDADES 2 Y 3

- Componentes, estado, estilos, navegación.
- Todo se resuelve dentro de React.
- Si funciona en tu teléfono, funciona en todos.

UNIDAD 4

- Cámara, GPS, acelerómetro, archivos, vibración.
- Hace falta código nativo del otro lado.
- El usuario puede **decir que no**, y el hardware puede directamente no existir.

> Dos ideas nuevas y ninguna es de React: **el permiso** (el usuario autoriza) y **la disponibilidad** (el dispositivo tiene o no tiene ese sensor). Las dos hay que manejarlas en el código.

El camino completo

Tu código
`Camera.takePictureAsync()`

MÓDULO NATIVO
⇄

API del sistema operativo
AVFoundation (iOS) · CameraX (Android)

Escrito en JS

Lo que vos importás: una función o un componente de React, con tipos de TypeScript.

Escrito en Swift / Kotlin

El módulo que traduce esa llamada a la API del sistema. Es lo que hay que compilar.

Por eso hace falta compilar

Agregar una librería nativa no es agregar un paquete de npm: cambia el binario.

De dónde sale una capacidad nueva

1 · Expo SDK

Módulos mantenidos por Expo, versionados junto al SDK y probados en cada release. Es el primer lugar donde buscar.

`docs.expo.dev/versions/latest`

2 · La comunidad

Todo lo demás. El directorio filtra por plataforma, mantenimiento y compatibilidad con Expo Go.

`reactnative.directory`

3 · Un módulo propio

Cuando no existe. Se escribe con `create-expo-module` en Swift y Kotlin. Fuera del alcance de la materia, pero conviene saber que se puede.

> Regla práctica para el TP integrador: **si está en el SDK de Expo, usá eso**. Se actualiza solo cuando subís de SDK y no te deja a mitad de camino en noviembre.

Antes de instalar: cuatro preguntas

¿En qué plataformas corre?

Arriba de cada página de la documentación están los badges Android · iOS · Web. Varios sensores dicen iOS (device only): en el simulador no andan.

¿Corre en Expo Go?

El badge Included in Expo Go decide si la vas a poder probar el primer día o si necesitás una development build.

¿Está viva?

Fecha del último commit, issues abiertos y, sobre todo, si declara soporte de la Nueva Arquitectura. Sin eso no corre en SDK 55 o superior.

¿Qué permisos arrastra?

Una librería puede agregar permisos al manifiesto sin que los pidas. Apple rechaza apps que declaran permisos que no usan.

`npx expo install`, no `npm install`

LO QUE HACE NPM

```
npm install expo-camera
# instala la última versión publicada
# → puede no ser compatible con tu SDK
```

LO QUE HACE EXPO

```
npx expo install expo-camera
# consulta qué versión corresponde al SDK 57
# instala esa, y avisa si algo quedó desfasado
```

Vale también para librerías de la comunidad: `npx expo install react-native-maps`. Expo mantiene una tabla de versiones compatibles por SDK y la usa para elegir.

`npx expo install --check` revisa el `package.json` entero y te dice qué está fuera de rango. Corrélo antes de entregar.

Qué de esta unidad corre en Expo Go

Sí — se prueba el primer día

- `expo-camera`
- `expo-image-picker`
- `expo-location`
- `expo-sensors` · `expo-haptics`
- `expo-file-system`
- `react-native-maps`
- Notificaciones **locales**

No — hace falta development build

- Notificaciones **push** DESDE SDK 53 EN ANDROID
- `expo-maps` ALPHA
- Cualquier librería sin el badge de Expo Go
- Cualquier config plugin que cambie el proyecto nativo

> Planificá el TP integrador con esto a la vista: si tu app necesita push, la development build tiene que estar hecha **antes** de la semana en que la vayas a usar, no el día anterior a la entrega.

UNIDAD 4 · PARTE 2

Generación nativa continua

Configurar el proyecto nativo sin abrirlo nunca.

El problema que resuelve

REACT NATIVE A MANO

- Editás `AndroidManifest.xml`, `Info.plist`, `build.gradle`, `Podfile`.
- Esos archivos viven en el repositorio.
- Actualizar de versión es resolver conflictos a mano en archivos que casi nadie del equipo entiende.

CNG CON EXPO

- Editás **solo** `app.json`.
- `android/` e `ios/` no están en el repositorio: se generan.
- Actualizar de SDK es cambiar un número y volver a generar.

CNG = Continuous Native Generation. La idea de fondo: el proyecto nativo es un **artefacto de build**, como `node_modules`. No se versiona, se reconstruye.

Cómo se genera



Cada librería aporta su config plugin: una función JS que sabe qué tocar del proyecto nativo.

Cómo se genera

app.json
lo único que escribís

CONFIG PLUGINS

android/ · ios/
generados

Permisos

`AndroidManifest.xml` · `Info.plist`

Dependencias nativas

`build.gradle` · `Podfile`

Recursos

Íconos, sonidos, claves de API, esquemas de URL

Un config plugin en `app.json`

```
{
  "expo": {
    "name": "Mi App",
    "slug": "mi-app",
    "plugins": [
      "expo-router",
      [
        "expo-camera",
        {
          "cameraPermission": "Sacamos la foto del producto que vas a publicar.",
          "microphonePermission": "Grabamos la nota de voz que adjuntás.",
          "recordAudioAndroid": true
        }
      ]
    ]
  }
}
```

Un string suelto es el plugin con su configuración por defecto. Un array es `["nombre", { opciones }]`. Los textos de permiso son los que **ve el usuario** en el cartel del sistema.

Cuándo hay que volver a compilar

Basta con recargar

- Cualquier cambio en tus componentes.
- Estilos, navegación, estado.
- Agregar una librería que sea solo JavaScript.

Fast Refresh lo aplica en el momento.

Hay que rearmar el binario

- Instalar una librería con código nativo.
- Agregar o cambiar un config plugin.
- Tocar permisos, íconos, nombre o esquema de URL.

`npx expo prebuild --clean` y una build nueva.

Síntoma típico: agregaste el plugin, recargaste, y el permiso sigue sin aparecer. **No es un bug tuyo** — el manifiesto viejo sigue en el binario instalado.

Si abrís `android/` a mano

Qué pasa

El proyecto pasa a bare: los directorios nativos ahora son tuyos y hay que versionarlos.

Qué perdés

El próximo `prebuild` pisa tus cambios. Y el salto de SDK deja de ser automático: pasa a ser una migración manual.

> Casi nunca hace falta. Si una librería pide un cambio nativo que no tiene plugin, se escribe un **config plugin propio** —es un archivo JS— antes de romper el modelo.

UNIDAD 4 · PARTE 3

Permisos

Declarar, pedir, y sobrevivir a que el usuario diga que no.

UNIDAD 4 · PARTE 3 · PERMISOS

Dos momentos distintos

1 · Declaración — cuando compilás

El binario dice qué permisos puede llegar a pedir. Va en `app.json` y termina en el manifiesto.

Si no está declarado, el pedido en tiempo de ejecución falla sin mostrar nada.

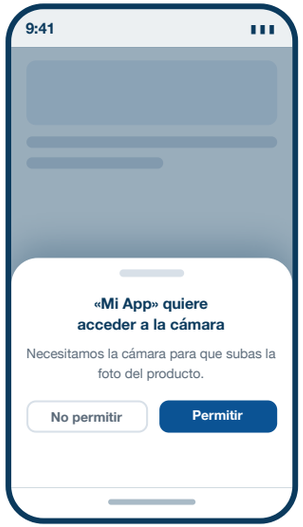
2 · Solicitud — cuando la app corre

El usuario ve un cartel del sistema y decide. Lo dispara tu código, en el momento que vos elijas.

Si no lo pedís, tenés el permiso declarado y la API igual falla.

> Los dos son necesarios. La mitad de los «no me anda la cámara» de la cursada son uno de estos dos pasos faltando.

Los estados de un permiso



El cartel lo dibuja el sistema operativo, no vos

`undetermined`

Todavía no se preguntó. Es el estado al instalar.

`granted`

Concedido. La API funciona.

`denied`

Rechazado. Y ojo con `canAskAgain`: si es `false`, el cartel **no vuelve a aparecer nunca**. La única salida es que el usuario lo cambie desde Ajustes.

El patrón moderno: un hook por permiso

```tsx
import { CameraView, useCameraPermissions } from 'expo-camera';
import { View, Text, Button } from 'react-native';

export default function Pantalla() {
  const [permiso, pedirPermiso] = useCameraPermissions();

  if (!permiso) return <View />;                          // 1 · resolviendo

  if (!permiso.granted) return (                          // 2 · no lo tenemos
    <View>
      <Text>Necesitamos la cámara para la foto del producto.</Text>
      <Button title="Dar acceso" onPress={pedirPermiso} />
    </View>
  );

  return <CameraView style={{ flex: 1 }} facing="back" />;   // 3 · adelante
}
```

Siempre son tres ramas, no dos. Saltear la primera es el bug más común: por un instante `permiso` es `null` y la pantalla parpadea con el mensaje de rechazo.

La misma idea, sin hook

CON HOOK — DENTRO DE UN COMPONENTE

```
const [permiso, pedir] = useCameraPermissions();

if (permiso?.granted) { /* … */ }
```

CON FUNCIONES — EN CUALQUIER LADO

```
import * as Location from 'expo-location';

async function ubicar() {
  const { status } =
    await Location.requestForegroundPermissionsAsync();

  if (status !== 'granted') return null;

  return Location.getCurrentPositionAsync({});
}
```

Las funciones vienen siempre de a pares: `get…PermissionsAsync()` consulta sin molestar al usuario, `request…PermissionsAsync()` muestra el cartel. Consultá primero.

iOS: el texto es obligatorio

```
{
  "expo": {
    "ios": {
      "infoPlist": {
        "NSCameraUsageDescription": "Sacamos la foto del producto que vas a publicar.",
        "NSLocationWhenInUseUsageDescription": "Mostramos los locales cercanos a donde estás."
      }
    }
  }
}
```

Si falta

La app **crashea** al pedir el permiso. No devuelve error: se cierra.

Si es genérico

Apple rechaza la publicación. «This app needs camera access» no alcanza.

> Los config plugins ya escriben estos strings —por eso `cameraPermission`—, pero el texto por defecto es genérico: reescribilo.

Android: agregar y también quitar

```
{
  "expo": {
    "android": {
      "permissions": ["android.permission.CAMERA"],
      "blockedPermissions": [
        "android.permission.RECORD_AUDIO"
      ]
    }
  }
}
```

`permissions`

Agrega al manifiesto. Casi nunca hace falta: las librerías declaran lo suyo.

`blockedPermissions`

Saca lo que una librería metió y vos no usás. Es el que más se usa.

Google Play te pide justificar cada permiso sensible. Un `RECORD_AUDIO` que entró de arrastre y no usás es una razón para que te rechacen.

Manifiestos de privacidad de Apple

Qué son

Un archivo `PrivacyInfo.xcprivacy` dentro del proyecto iOS donde declarás **por qué** tu app usa ciertas APIs sensibles: acceso a archivos, espacio en disco, hora del sistema, preferencias del usuario.

Obligatorio para publicar en la App Store.

Cómo se resuelve en Expo

Se declara en `app.json` bajo `ios.privacyManifests` y CNG lo genera. Las librerías del SDK ya aportan sus propias razones.

Aparece recién al preparar la entrega a la tienda: lo retomamos en la Unidad 7.

> No confundirlo con los strings de `infoPlist`. Aquellos son **para el usuario**; este es **para Apple**.

Cómo se pide bien

Mal

- Pedir todos los permisos al abrir la app.
- Mostrar el cartel sin ningún contexto previo.
- Dar por hecho que dijo que sí.
- Dejar la pantalla en blanco si dijo que no.

Bien

- Pedirlo cuando el usuario aprieta «Sacar foto».
- Explicar antes, con tu propia interfaz, para qué es.
- Manejar las tres ramas del hook.
- Ofrecer un camino alternativo: elegir de la galería, escribir la dirección a mano.

Se evalúa en el TP integrador: **la app tiene que seguir siendo usable si el usuario rechaza todo**. No tiene que hacer todo, pero no puede romperse.

UNIDAD 4

Documentación

- `docs.expo.dev/workflow/using-libraries`
- `docs.expo.dev/workflow/continuous-native-generation`
- `docs.expo.dev/guides/permissions`
- `docs.expo.dev/guides/apple-privacy`